

Binary Search Tree (BST) & Operations Data Structures

> **Definition:** A **Binary Search Tree (BST)** is a binary tree with the ordering property: for every node `X`, all keys in `X`'s left subtree are strictly **less than** `X.key`, and all keys in `X`'s right subtree are strictly **greater than** `X.key`.

1. Detailed Technical Explanation

[50]

1. Search Operation in BST

- Compare search key `K` with `root->data`:

- If `K`

2. Insertion Operation

- Traverse tree following BST property until reaching a `NULL` pointer, then insert new node as a leaf.

3. Deletion Operation in BST (Three Cases):

- Case 1: Node is a Leaf (0 Children):** Simply delete the node and set parent's pointer to `NULL`.

2. **Case**

2. Complete Executable C Implementation of BST

```
```c
```

```
#include
```

```
struct Node {
```

```
int data;
```

```
struct Node* createNode(int key) {
```

```
struct Node
```

// 1. BST Insert

struct t

// Find minimum node (Inorder Successor helper)

struct t

// 2. BST Delete

struct t

if (key < root->data)

root->l

void inorder(struct Node\* root) {

if (root

---

### 3. Time and Space Complexities

| Operation | Average Case | Worst Case (Skewed) |

| :--- | :

---

### 4. Quick Recall Flow

'''

BST P  
succes